
齐鲁理工学院

QILU INSTITUTE OF TECHNOLOGY

本科毕业设计（论文）



题 目 基于 Python 的木质板材表面缺陷系统的
 设计与实现

姓 名 栾艳康 学号 241053011120

二级学院 计算机与信息工程学院

专 业 计算机科学与技术

指导教师 刘荣 职称 讲师

齐鲁理工学院教务处制

目录

1 绪论.....	6
1.1 研究背景.....	6
1.2 研究现状.....	6
1.2.1 国内研究现状	6
1.2.2 国外研究现状	7
1.3 论文组织结构.....	7
2 关键技术.....	8
2.1 Python	8
2.2 Flask.....	8
2.3 YOLO 目标检测算法.....	9
2.4 SQLite 与 SQLAlchemy.....	9
3 系统分析.....	9
3.1 可行性研究.....	9
3.1.1 经济可行性	9
3.1.2 操作可行性	10
3.1.3 技术可行性	10
3.2 需求分析.....	10
3.3 系统功能分析.....	11
4 系统设计.....	12
4.1 系统结构设计.....	12
4.2 系统顺序图设计.....	14
4.2.1 登录顺序图	14
4.2.2 缺陷检测顺序图	14
4.2.3 历史追溯与报表顺序图	15

4.2.4 销售出库顺序图	16
4.3 数据库设计.....	16
4.3.1 数据库 E-R 图设计.....	16
4.3.2 数据库表设计	18
5 系统实现.....	20
5.1 用户功能模块的实现.....	20
5.1.1 用户登录界面	20
5.1.2 检测历史查询界面.....	21
5.1.3 日志报表查看界面.....	22
5.1.4 销售与库存管理界面	23
5.1.5 缺陷图谱与帮助中心界面.....	24
5.2 管理员功能模块的实现.....	25
5.2.1 用户管理界面	25
5.2.2 系统设置与模型切换界面.....	26
5.3.1 数据获取与预处理.....	27
5.3.2 系统检测与可视化分析	29
6 系统测试.....	31
6.1 测试目的.....	31
6.2 测试环境.....	31
6.3 测试用例.....	31
6.3.1 正常功能测试截图.....	32
6.3.2 异常功能测试截图.....	33
6.4 测试结论.....	33
7 总结与展望.....	33
参考文献.....	34

致谢.....34

基于 Python 的木质板材缺陷检测系统的设计与实现

计算机科学与技术专业 栾艳康

指导教师 刘荣

摘要：为了克服传统木材表面缺陷检测时人工识别速度慢、检测结果难保存、质量分级和库存管理之间缺少联系等缺点，本文设计并实现了以 Python 为基础的木材表面缺陷智能检测系统。系统主要使用 Python 进行开发，用 Flask 框架做后端业务处理，用 SQLite 数据库存储用户的个人信息、检测记录、工作日志、库存数据和销售记录，使用 YOLO 目标检测算法对木材表面缺陷进行识别。系统使用加载好的权重文件对上传的图片进行推理，得到缺陷类别、置信度和检测结果图，最后根据裂纹、死节、活节等缺陷类型进行板材等级判定。系统实现了用户登录、图片上传检测、摄像头采集检测、检测历史查询、日志报表统计、Excel 数据导出、库存联动、销售出库、缺陷图谱展示、管理员用户管理、参数设置、模型切换等主要功能。测试结果表明，该系统可以较好地完成木材表面缺陷检测以及业务数据管理工作，有较好的实用价值和教学演示价值。

关键词：Python 木材缺陷检测 YOLO Flask Web 系统 库存销售联动

Design and Implementation of a Python-Based Surface Defect Detection System for Wood Panels

Computer Science and Technology Luan Yankang

Supervisor Liu Rong

Abstract: To address the low efficiency of manual wood panel surface defect inspection, the difficulty of preserving inspection records, and the weak connection between quality grading and inventory management, this thesis designs and implements a Python-based web defect detection system for wood panels. The system uses Flask for back-end service control, SQLite and SQLAlchemy for data persistence, and a YOLO detection model for identifying common surface defects such as cracks, dead knots and live knots. After users upload images or capture images through a camera, the system returns defect categories, confidence values, result images and grading results, and stores the corresponding inspection records. In addition, the system provides login authentication, history query, log statistics, report export, inventory linkage, sales outbound management, user management and model switching functions. Functional testing shows that the system can complete the main processes of defect detection, result tracing and business data management, and has practical value for teaching demonstration and graduation project implementation.

Keywords: Python; wood panel defect detection; YOLO; Flask; Web system; inventory and sales linkage

1 绪论

1.1 研究背景

近年来，随着家具制造、建筑装饰和木质板材加工行业的持续发展，板材质量检测在生产管理中的作用逐渐增强。木质板材表面容易出现裂纹、死节、活节等缺陷，这些缺陷不仅会影响板材外观，还可能降低板材力学性能和后续加工利用率。若缺陷不能被及时发现并记录，容易造成质量分级不准确、产品流转信息不完整以及后续销售管理依据不足等问题。因此，对木质板材表面缺陷进行及时、准确的检测，具有一定的工程应用价值和现实意义。

目前，在部分板材加工和质量检查场景中，表面缺陷识别仍然较多依赖人工观察完成。质检人员通常根据经验对板材表面纹理、裂痕、节疤等情况进行判断，这种方式虽然操作简单，但在批量检测任务中效率较低，且检测结果容易受到人员经验、工作状态、光照条件和视觉疲劳等因素影响。同时，传统检测记录常以纸质表单或零散文件保存，检测结果与板材等级、库存变化、销售出库等业务数据之间缺少有效联系，后续进行质量追溯和统计分析时不够方便。

针对上述问题，引入计算机视觉和目标检测技术，构建面向木质板材表面缺陷识别的 Web 检测系统，是一种较为可行的解决思路。基于此，本文采用 Python 作为主要开发语言，结合 Flask Web 框架、SQLite 数据库和 YOLO 目标检测模型，设计并实现一套木质板材缺陷检测系统。系统能够对用户上传或摄像头采集的板材图像进行检测，输出缺陷类别、置信度、结果图和等级判定结果，并将检测历史、日志报表、库存变化和销售出库等数据统一管理。通过该系统的设计与实现，可以在一定程度上降低人工检测负担，提高检测结果保存和业务追溯的便利性，为木质板材质量检测的信息化管理提供参考。

1.2 研究现状

1.2.1 国内研究现状

国内关于木质板材表面缺陷识别的研究近几年推进较快，研究重点逐渐从传统人工观察和简单图像处理转向机器视觉、深度学习和目标检测方法。朱咏梅等人将注意力机制与可变形卷积网络应用于木质板材瑕疵识别，说明深度学习方法能够提升模型对复杂纹理背景下缺陷区域的表达能力[1]。吴志轩等人提出基于混合纹理特征的板材表面缺陷检测方法，从纹理特征提取角度提高缺陷识别效果[8]。这些研究表明，围绕板材表面纹理复杂、缺陷形态差异较大等问题，国内学者已经从特征提取和网络结构优化两个方向开展了较多探索。

在目标检测方法方面，陈宇针对木质板材表面缺陷在线检测任务研究了改进 YOLOX 算法[11]，何涵围绕木质板材封边缺陷检测对 YOLOv5 模型进行了改进[12]，易志浩则从机器视觉角度研究木质板材表面缺陷检测方法[13]。此外，徐浩等人从多特征融合角度研究在线板材表面缺陷检测方法[14]，凌嘉欣、谢永华将残差神经网络模型用于木质板材缺陷分类[15]。从这些研究可以看出，国内木质板材缺陷检测已经形成了较为明确的算法研究基础，研究内容主要集中在缺陷识别精度、复杂纹理适应能力、在线检测效率和模型结构改进等方面。

1.2.2 国外研究现状

国外在工业视觉检测领域起步较早，早期研究多依靠纹理、边缘、颜色等人工特征对材料表面进行分析，并结合支持向量机、决策树等传统机器学习方法完成缺陷分类。随着深度学习技术的发展，卷积神经网络、目标检测网络和图像分割模型逐渐成为工业表面缺陷检测的重要方法。相比传统特征方法，深度学习模型能够从大量样本中自动学习缺陷特征，在复杂背景、小尺度缺陷和多类别识别任务中具有更强的适应能力。

在工程应用方面，国外工业检测研究不仅关注缺陷识别准确率，也较为重视检测系统在生产现场中的部署条件、运行稳定性和数据追溯能力。例如，Deqiang 等人围绕板材检测中的远场涡流探头问题进行了研究，说明板材类对象的检测不仅涉及缺陷识别，还与检测设备、材料特性和现场应用条件有关[9]。对于木材加工、金属制造、纺织生产和电子制造等场景，视觉检测系统通常需要与采集设备、数据库和生产管理流程相结合，才能完成质量分级、异常报警、统计分析和历史追溯等任务。由此可见，缺陷检测系统的价值并不只体现在模型是否能够识别缺陷，还体现在检测结果能否被有效保存、管理和利用。

1.3 论文组织结构

本文共分为七章，具体结构安排如下。

（1）绪论。主要介绍课题研究背景、国内外研究现状以及论文整体组织结构，说明木质板材缺陷检测系统的研究意义。

（2）关键技术。主要介绍系统开发过程中使用的 Python、Flask、YOLO 目标检测算法和 SQLite 等相关技术，为后续系统设计和实现提供技术基础。

（3）系统分析。主要从经济可行性、操作可行性、技术可行性、需求分析和系统功能分析等方面对系统进行说明，明确系统需要解决的问题和实现范围。

（4）系统设计。主要包括系统总体结构设计、顺序图设计和数据库设计，说明系统各功能模块之间的关系以及主要数据表结构。

(5) 系统实现。主要介绍用户功能模块、管理员功能模块和算法检测模块的具体实现过程，包括登录、检测、历史记录、报表统计、库存销售、用户管理、模型切换等功能。

(6) 系统测试。主要通过设计测试用例对系统主要功能、异常处理情况和运行结果进行验证，判断系统是否满足预期设计要求。

2 关键技术

2.1 Python

Python 语法相对简洁，第三方库生态较完善，适合用于 Web 服务开发、图像处理和深度学习模型调用等任务。本文所设计的木质板材缺陷检测系统需要同时完成页面请求处理、数据库操作、图片文件保存、检测模型调用和报表数据整理等工作，因此选择 Python 作为主要开发语言。

在本系统中，Python 的作用主要体现在三个方面。第一，后端程序通过 Flask 路由函数接收用户登录、图片上传、历史查询、销售出库和后台设置等请求，并返回相应页面或 JSON 数据。第二，系统通过 SQLAlchemy 完成用户信息、检测历史、工作日志、库存和销售记录等数据的增删改查。第三，检测模块使用 Ultralytics 接口加载 YOLO 权重文件，对木质板材图片进行推理，并将检测结果转换为缺陷类别、置信度和板材等级。由此可见，Python 在本系统中不仅承担基础业务处理工作，也承担模型调用和数据流转的连接作用。

2.2 Flask

Flask 是 Python Web 开发中常用的轻量级框架，其核心功能包括路由映射、请求处理、模板渲染和响应返回。与功能较完整的大型框架相比，Flask 对项目结构的约束较少，开发者可以根据系统规模和业务需求灵活组织代码。对于毕业设计中的中小型 Web 系统而言，Flask 既能够满足页面访问和业务处理需求，又便于与数据库操作、文件上传和模型推理等功能进行集成。

在本系统中，Flask 主要承担后端业务控制作用。系统通过不同路由函数分别处理用户登录、图片上传检测、摄像头检测、历史记录查询、日志报表统计、销售出库、用户管理和系统设置等请求；通过模板渲染将检测页面、历史页面、报表页面和后台管理页面返回给浏览器；通过 JSON 响应将检测结果、提示信息和局部操作结果返回给前端。同时，系统结合 Flask-Login 实现用户登录状态维护和权限控制，使普通用户与管理层在同一套 Web 服务中拥有不同的功能访问范围。

2.3 YOLO 目标检测算法

YOLO (You Only Look Once) 是一种典型的单阶段目标检测算法, 最初由 Redmon 等人于 2016 年提出, 其核心思想是将目标检测任务视为回归问题, 在一次前向推理过程中同时完成目标定位与类别判断, 从根本上改变了传统两阶段检测方法先提取候选区域再进行分类的处理模式。经过从 YOLOv1 到 YOLOv8 的多个版本迭代, YOLO 系列算法在检测精度、推理速度和模型鲁棒性方面持续提升, 在保持实时检测能力的同时对小目标、多类别及复杂背景场景均表现出较强的适应性, 目前已广泛应用于工业缺陷检测、安防监控和医学图像分析等领域, 是当前工业视觉检测研究中使用较为普遍的算法之一。

在本系统中, 后端检测函数会先从数据库读取管理员设置的置信度阈值和当前模型文件路径, 再将保存后的木质板材图片传入 Ultralytics 提供的 YOLO 推理接口完成检测, 得到缺陷类别、置信度和带检测框的结果图。模型返回结果后, 系统将英文类别名称转换为中文缺陷名称 (如 Crack→裂纹、Dead Knot→死节、Live Knot→活节), 并按照预设的等级判定规则生成板材等级——无明显缺陷判定为 A 级, 检测到活节判定为 B 级, 检测到裂纹或死节判定为 C 级。这样处理后, 目标检测结果可直接服务于结果展示、检测记录保存、库存更新和销售出库等后续业务, 使模型推理与系统数据管理形成有效衔接。

2.4 SQLite 与 SQLAlchemy

SQLite 是一种轻量级关系型数据库, 具有部署简单、无需单独安装数据库服务、数据以文件形式保存等特点。对于毕业设计和本地演示类系统来说, SQLite 能够满足基本的数据持久化需求, 同时可以降低系统安装、配置和迁移的复杂度。

在本系统中, SQLite 主要用于保存用户信息、检测历史、工作日志、销售记录、系统参数和库存数据。后端程序结合 SQLAlchemy 对数据库进行操作, 将数据表映射为 Python 类, 并在登录认证、检测记录保存、历史查询、报表统计、库存联动和销售出库等功能中完成数据读取与写入。通过这种方式, 系统能够把模型检测结果和业务管理数据统一保存, 为后续查询、统计和追溯提供支持。

3 系统分析

3.1 可行性研究

3.1.1 经济可行性

从经济角度看, 本系统在硬件方面不需要采购专用检测设备, 使用普通配置的个人计算机即可完成系统部署和运行, 不涉及较高的硬件购置费用。在软件方面, 系统开发所采用的

Python、Flask、SQLite、Ultralytics、OpenCV、Bootstrap 和 Chart.js 等工具和组件均为开源或免费项目，无需支付商业软件授权或许可证费用，从而有效降低了软件成本。数据库采用 SQLite 文件存储方式，不需要单独购买和部署商业数据库服务器，进一步减少了基础设施投入。检测所需样本图片可通过已有数据集和系统上传功能完成准备，不需要专门采购。综合来看，本系统的直接开发费用较低，主要成本集中在开发时间的人力投入上，符合毕业设计阶段的经济条件，经济可行性较好。

3.1.2 操作可行性

从操作角度看，本系统采用 B/S 架构，用户只需通过浏览器即可访问所有功能页面，客户端无需安装任何额外软件，降低了部署和维护的复杂度。系统界面基于 Bootstrap 框架构建，采用统一的侧边导航布局，各功能模块入口清晰，操作流程比较直观。普通用户经过简单引导即可完成图片上传、检测结果查看、历史记录查询和报表浏览等常规操作；管理员可在独立的后台页面中完成用户管理、阈值设置和模型切换等维护工作，各角色的功能入口和操作边界均有明确区分。检测页面同时支持本地上传和摄像头采集两种输入方式，能够适应不同的使用场景。整体而言，系统操作门槛较低，对用户的技术背景要求不高，具备较好的操作可行性。

3.1.3 技术可行性

从技术角度看，本系统在软件层面选用了 Python、Flask、SQLite、Ultralytics YOLO 和 OpenCV 等较成熟的开源技术，相关文档和社区资源丰富，开发过程中所需的技术参考材料获取方便，技术风险较低。在硬件方面，系统在普通消费级计算机上即可完成 YOLO 模型推理和业务逻辑处理，不需要配备专用服务器或加速设备，对运行环境的要求相对宽松。从已有研究基础来看，YOLO 系列模型在工业缺陷检测领域已有大量成功应用，Flask 用于构建中小型 Web 服务的可行性也已得到充分验证。当前系统已在本地环境中完成用户登录、图片检测、结果保存、库存更新和销售记录写入等主要业务流程的开发与调试，说明所选技术路线能够支撑本系统核心功能的实现。综上，本系统的技术可行性较好。

3.2 需求分析

根据系统实际的实现情况，本文将系统使用者划分为普通用户和管理员两种角色。普通用户主要进行登录认证、缺陷检测、历史记录查看和报表浏览等常规操作；管理员在此基础上还需承担用户账号管理、检测参数配置和检测模型切换等后台维护工作。这一角色划分与项目中的权限控制逻辑保持一致，能够为后续功能分析提供清晰的边界。

在检测业务方面，系统需要支持单张或多张图片的本地上传检测，以及通过浏览器调用摄像头进行实时采集检测。检测完成后，系统应返回缺陷类别、置信度和板材等级，并将结果图保存到服务器指定目录。在数据管理方面，系统需要记录每次检测的文件名、操作时间、操作人员、结果图路径和等级信息，并支持按缺陷类型进行关键词检索和分页查询。

在统计分析方面，系统需要汇总今日检测数量、累计工作时长、缺陷类别占比和近七天检测趋势等数据，并以图表形式呈现，同时支持导出 Excel 格式的报表文件。在业务联动方面，检测完成后系统需要根据判定等级自动更新对应的库存数量，销售出库时需要校验库存余量，完成库存扣减和销售记录写入，从而形成检测、入库到出库的完整业务闭环。此外，系统还需要提供缺陷图谱展示页面，帮助用户了解常见缺陷的外观特征和等级判定标准。

3.3 系统功能分析

从系统使用角色和业务流程来看，本系统主要包括普通用户功能、管理员功能和公共支撑功能三部分。普通用户负责完成缺陷检测、检测历史查询、日志报表查看、销售出库、缺陷图谱浏览和个人信息维护；管理员在普通用户功能的基础上，增加用户管理、系统参数设置、模型切换和日志查看等维护功能。两个角色在统一登录认证和权限控制基础上共享检测、历史、报表和库存等业务数据。

系统功能结构以缺陷检测为核心展开。检测模块接收本地上传或摄像头采集的板材图片，调用模型完成识别和等级判定；历史记录模块保存检测结果并支持追溯查询；日志报表模块对检测数量、工作时长和缺陷类别进行统计；库存与销售模块根据等级结果完成库存更新和出库记录保存。管理员模块主要为上述业务提供账号、参数和模型文件维护能力。

按照页面入口细分，系统包含登录、仪表盘、缺陷检测、历史记录、日志报表、数据导出、销售管理、帮助中心、用户管理和系统设置等功能页面。普通用户只访问与检测和业务操作相关的页面，管理员可额外进入后台维护页面。该功能划分能够对应系统实际代码中的路由、模板和数据库操作，也为后续系统设计图和实现章节提供依据。

就普通用户而言，其主要的操作有登录认证、缺陷检测、历史记录查询、日志报表查看、销售管理、帮助中心浏览、个人中心维护等。普通用户和系统之间用例图如下图 3-1 所示。

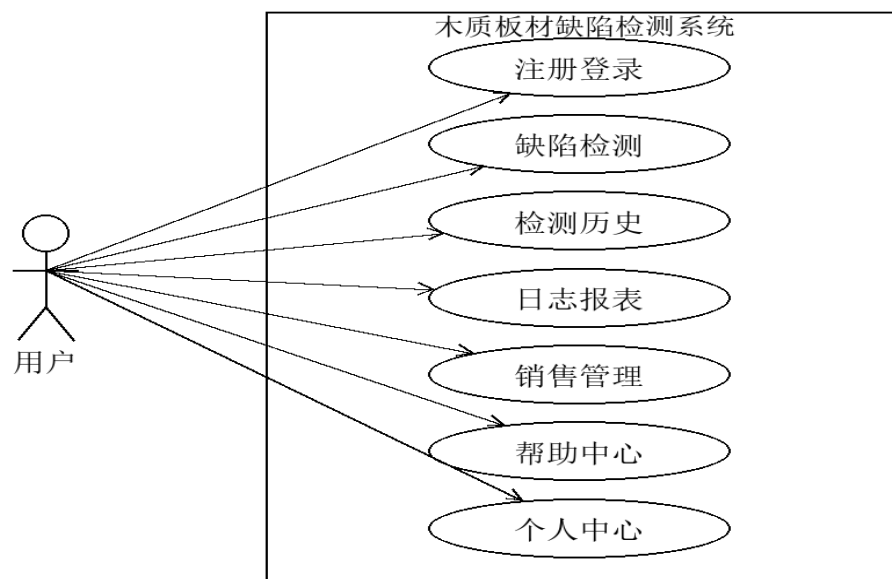


图 3-1 普通用户用例图

从管理员的职责范围来说，在普通用户的权限之上还拥有用户管理、系统设置、模型切换以及日志查看这些管理权限。管理员和系统之间的用例图如图 3-2 所示。

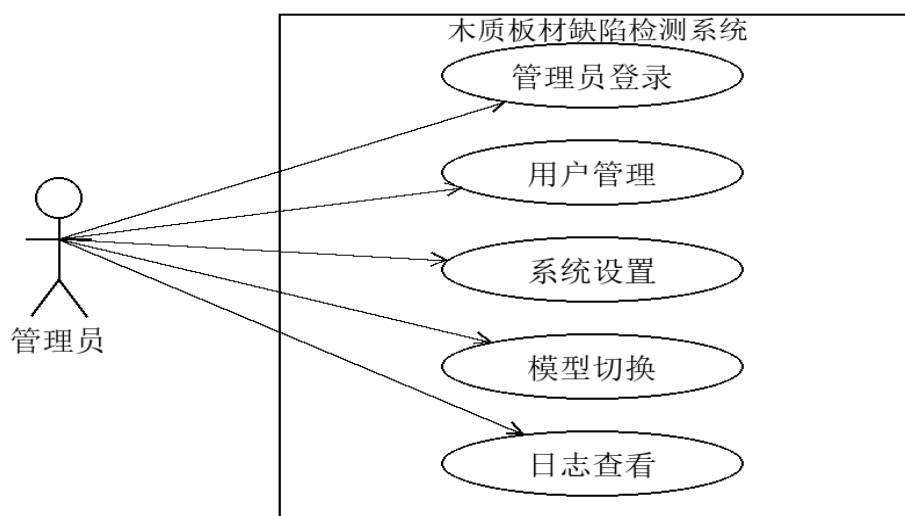


图 3-2 管理员用例图

4 系统设计

4.1 系统结构设计

该系统采用 B/S 架构，用户通过浏览器浏览各种功能页面，服务器端做业务处理、模型推理、数据库读写等工作。该架构对于客户端环境要求不高，在本地或者局域网内可以很快地进行部署和演示。系统的总体结构图如下图 4-1 所示。

从构成上来说，前端页面主要实现图片选择、检测结果展示、图表显示、表单提交的功能；后端服务主要是接收到请求之后执行业务逻辑、调用检测模型返回处理结果；数据库用来保存用户的、检测记录的、工作日志的、库存的、销售的等数据；算法模块主要是对木材表面缺陷进行识别。各个部分一起工作，可以共同支撑系统的主要业务功能。

系统启动时会自动创建上传目录和结果目录，然后进行数据库表的初始化。当系统中没有默认的管理员账号或者初始库存信息时，程序就会写入基础数据，从而保证系统第一次运行之后就可以完成登录、检测以及业务流程演示。

系统用统一布局模板来组织页面。用户登录之后，公共模板会承载起侧边导航、用户信息、消息提示以及功能入口的作用，各个业务页面在统一的框架中会以各自的页面内容来呈现出来。该种设计既保证了界面风格的统一，又利于以后页面的维护以及功能的扩展。

系统业务层主要是由后端路由函数组成的。不同的路由对应着不同的请求，即登录、检测、历史记录、日志报表、销售管理、用户管理、系统设置等请求，在接收到前端的数据之后进行权限判断、数据处理、模型调用、数据库提交等工作。

从数据流的角度来分析，系统主要有检测数据流、统计数据流和销售数据流这三个流程。检测数据流从图片上传开始，经过模型推理和等级判定之后写入检测历史并更新库存，统计数据流从检测记录、工作日志中提取数据生成报表和趋势图，销售数据流根据订单信息完成库存扣减并保存销售记录。三类数据流一起组成了系统的业务链路。

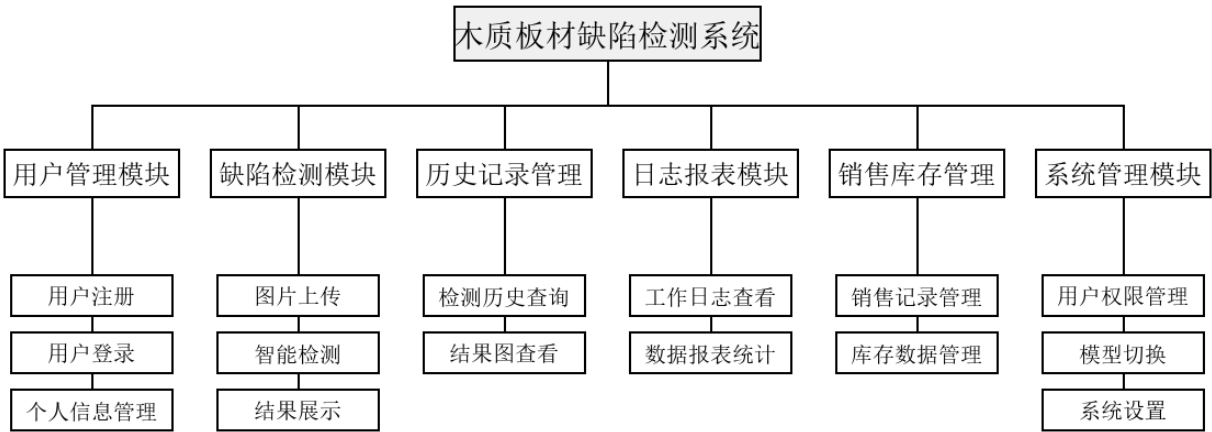


图 4-1 系统总体结构图

4.2 系统顺序图设计

4.2.1 登录顺序图

登录顺序图主要描述的是用户在登录过程中同前端页面、后端服务和数据库之间发生的交互关系。用户在登录页面输入账号、密码并选择身份入口之后，系统先查询用户的账号、密码以及角色是否符合要求，然后进行密码和角色的验证；验证成功之后会将用户的登录状态写入到数据库中，并且记录下用户的日志，如果验证失败，则会给出相应的提示信息。登录顺序图如下图 4-2 所示。

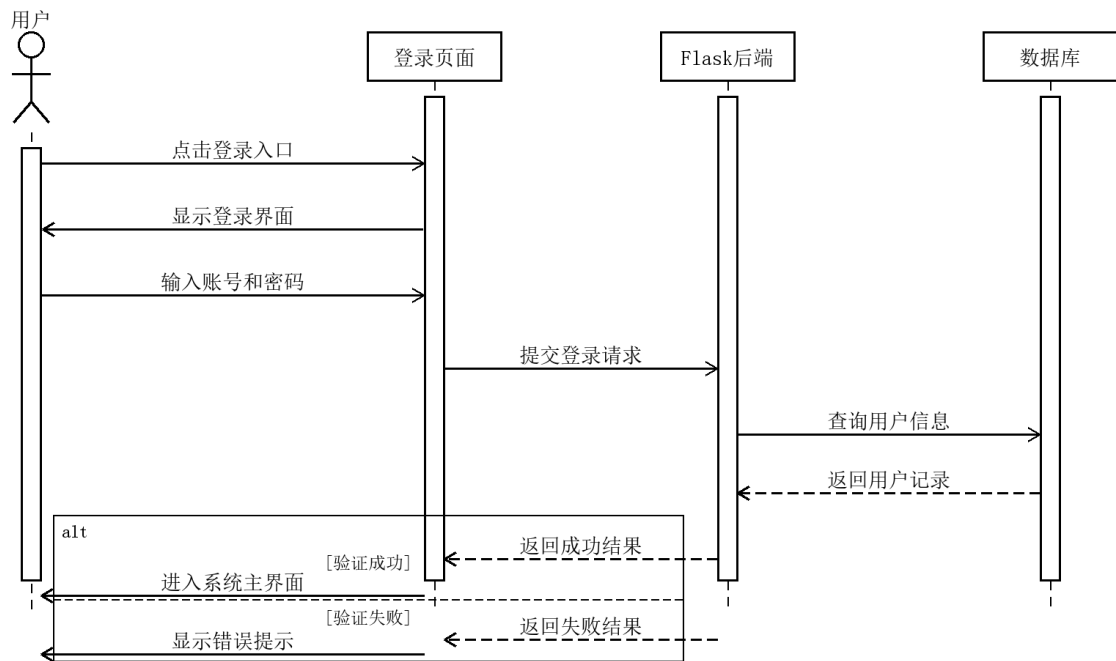


图 4-2 登录顺序图

4.2.2 缺陷检测顺序图

缺陷检测顺序图用来说明图片上传、模型推理、结果处理、数据保存之间调用关系。用户提交图片之后，后端先保存原始文件并读取当前检测参数，然后调用 YOLO 模型进行推理；系统根据模型输出得到检测结果图、缺陷类别、置信度和等级信息，最后将结果写入数据库。缺陷检测顺序图如下图 4-3 所示。

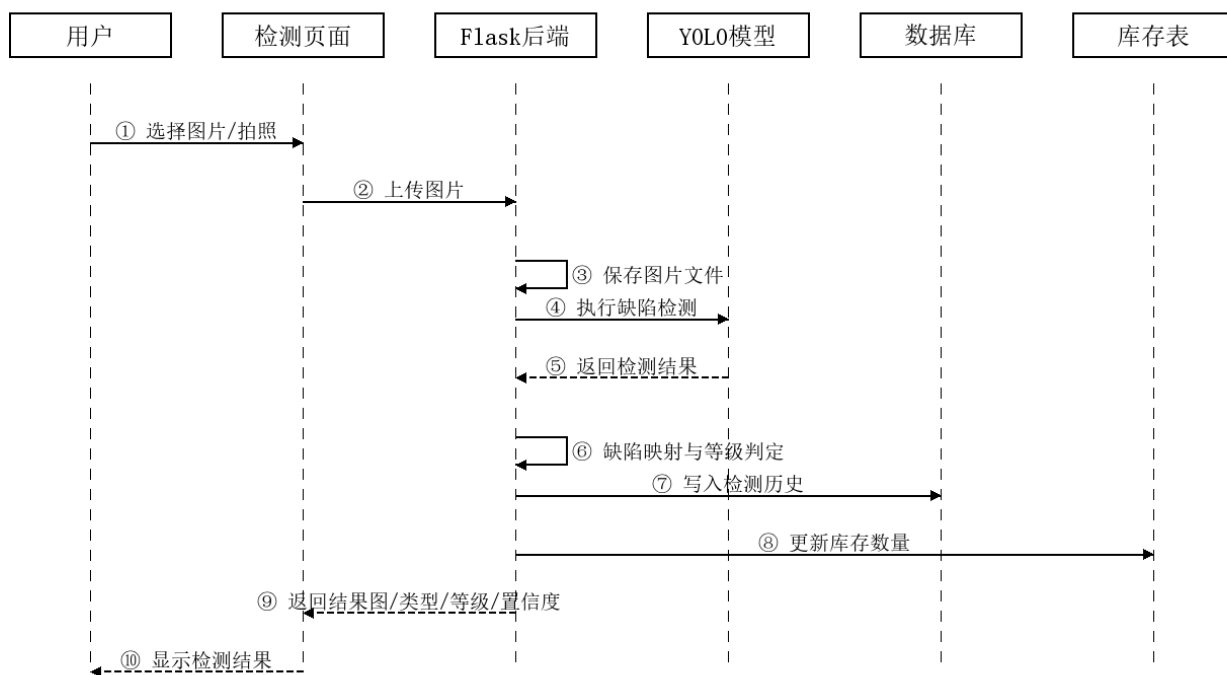


图 4-3 缺陷检测顺序图

4.2.3 历史追溯与报表顺序图

历史追溯和报表顺序图主要体现系统对于检测数据进行查询、统计、展示的过程。用户进入历史记录或者日志报表页面之后，系统就会按照当前的身份来确定数据的访问范围，然后依次完成分页查询、缺陷筛选、统计汇总以及图表渲染等工作，以此来实现检测结果的追溯以及运行情况的分析。历史追溯以及报表顺序图如图 4-4 所示。

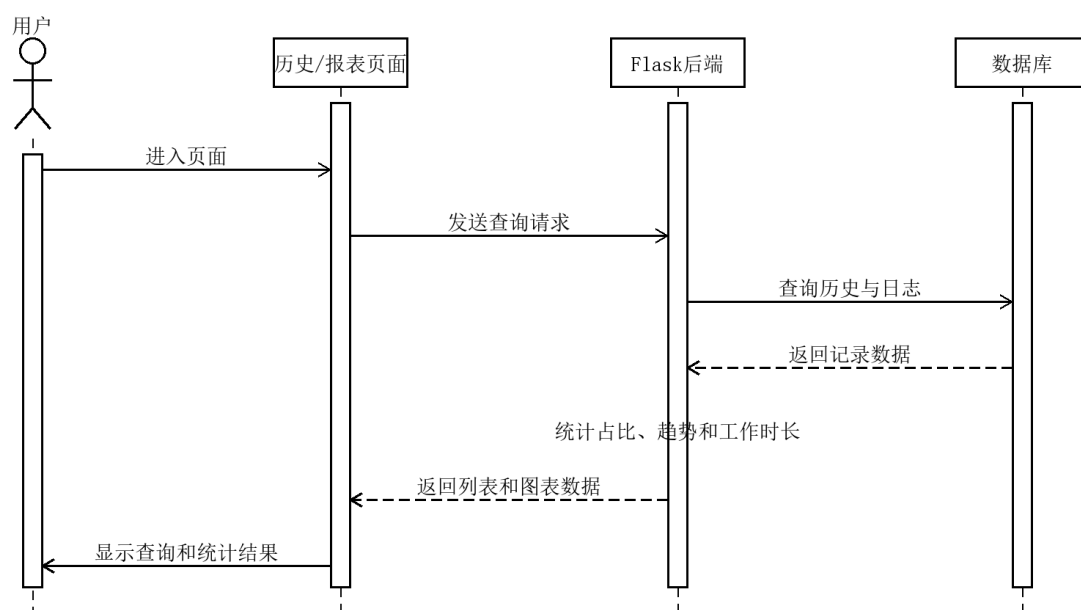


图 4-4 历史与报表顺序图

4.2.4 销售出库顺序图

销售出库顺序图用来描述订单提交和库存扣减之间业务处理的过程。用户在销售页面输入客户名称、板材等级、数量、金额之后，先查询对应等级的库存是否满足出库条件，满足出库条件就执行库存扣减和销售记录保存，否则提示信息并结束操作。销售出库顺序图如下图 4-5 所示。

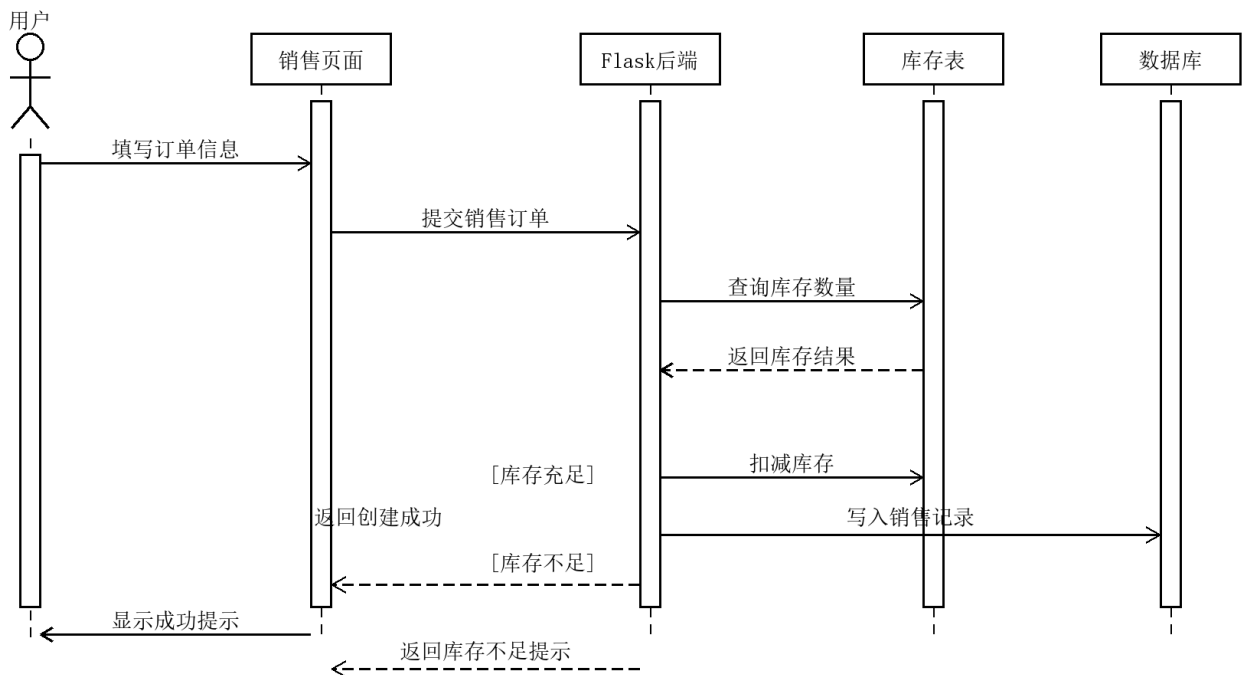


图 4-5 销售出库顺序图

4.3 数据库设计

4.3.1 数据库 E-R 图设计

数据库的 E-R 图用来表示系统中的主要实体、属性以及实体之间的联系。本系统根据实际数据库模型抽取出用户、检测历史、工作日志、销售记录、库存和系统设置六类实体。用户实体用于保存登录账号、密码哈希和角色信息；检测历史实体用于保存原始文件名、结果图、缺陷类型、置信度、检测时间和等级结果；工作日志实体用于记录用户登录、退出和累计工作时长；销售记录实体用于保存客户、等级、数量、金额和销售时间；库存实体用于记录不同等级板材的库存数量；系统设置实体用于保存当前模型路径和检测阈值等参数。主要实体属性如图 4-6 所示。

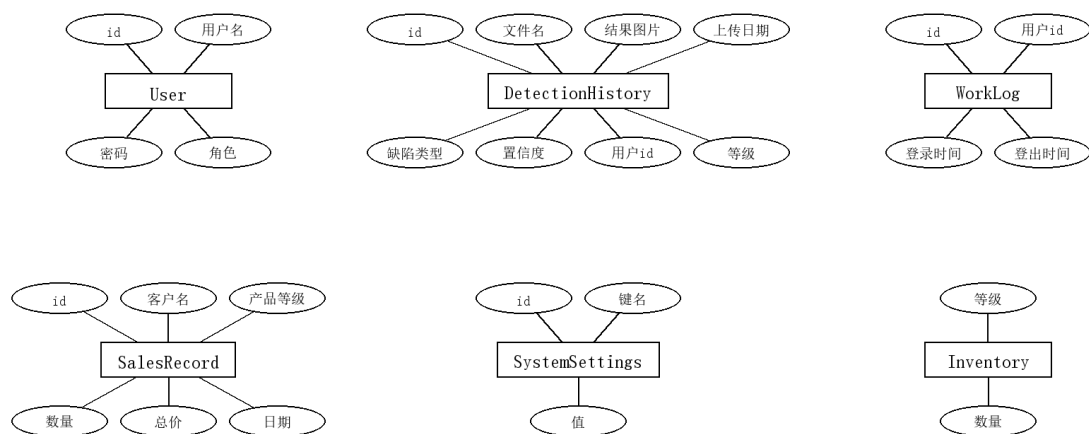


图 4-6 主要实体属性图

在实体关系上，一个用户可以产生多条检测历史记录，也可以对应多条工作日志记录，因此用户与检测历史、用户与工作日志均为一对多关系。检测历史中的等级结果会影响库存数量，库存又会被销售出库业务读取和扣减，因此库存与销售记录之间通过板材等级形成业务关联。系统设置实体为检测流程提供模型路径和阈值参数，不直接依赖某一条检测记录，但会影响后续检测结果生成。系统各个实体之间的联系如图 4-7 所示。

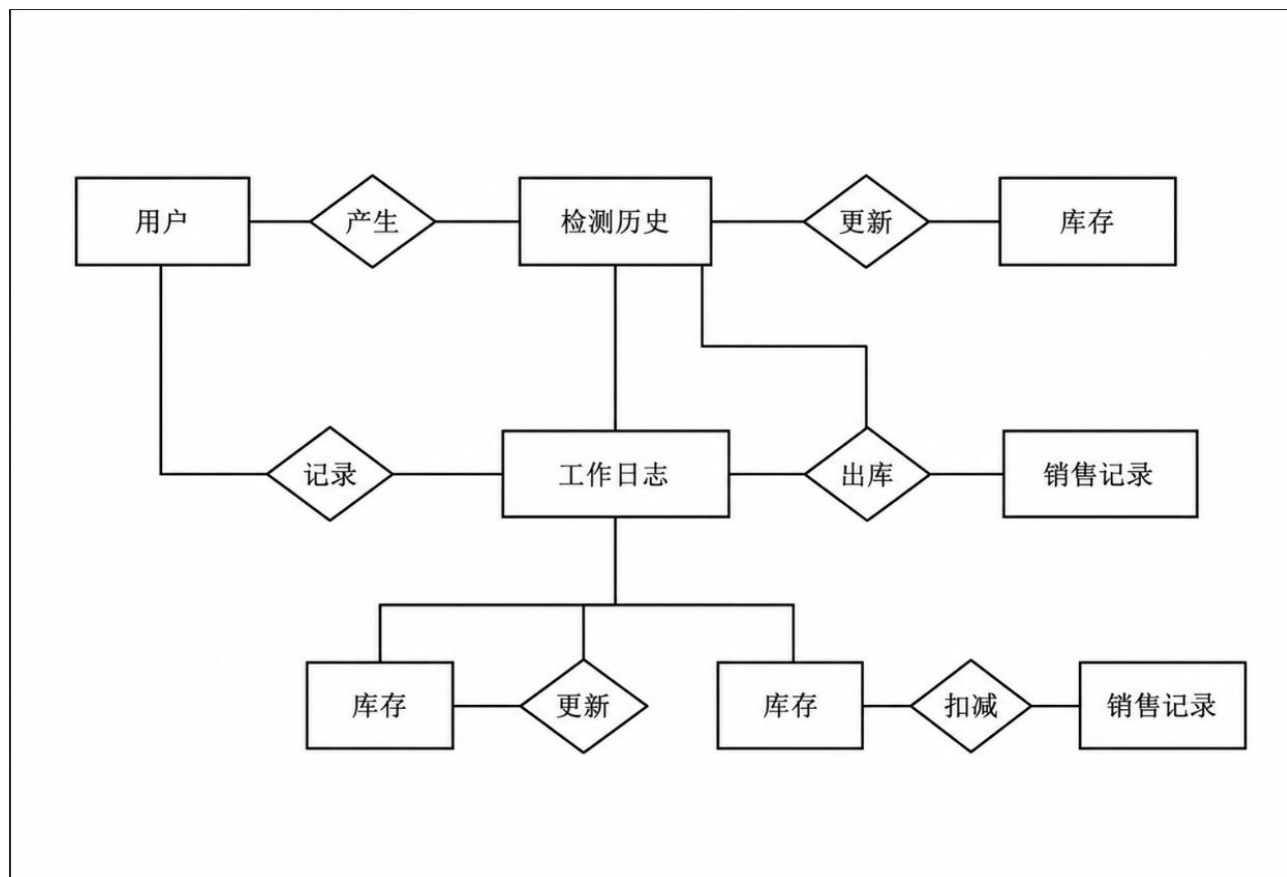


图 4-7 系统 E-R 图

4.3.2 数据库表设计

根据系统功能需求和 E-R 图设计结果，数据库主要包括用户表、检测历史表、工作日志表、销售记录表、系统设置表和库存表。各表分别承担账号管理、检测结果保存、工作时间记录、销售业务存储、参数设置和库存统计等任务。表结构设计既要与系统实际模型保持一致，也要说明字段含义、数据类型和业务用途。

表 4-1 用户表 User

字段名	类型	说明
id	Integer	主键，自增
username	String(50)	用户名，唯一，非空
password	String(255)	密码哈希值，非空
role	String(20)	角色，user/admin

表 4-2 检测历史表 DetectionHistory

字段名	类型	说明
id	Integer	主键，自增
filename	String(300)	原始文件名
result_image	String(300)	结果图文件名
upload_date	DateTime	上传检测时间
defect_type	String(100)	缺陷类型
confidence	Float	最高置信度
user_id	Integer	关联用户 ID
grade	String(10)	板材等级

表 4-3 工作日志表 WorkLog

字段名	类型	说明
id	Integer	主键，自增
user_id	Integer	操作用户 ID
login_time	DateTime	登录时间
logout_time	DateTime	退出时间

表 4-4 销售记录表 SalesRecord

字段名	类型	说明
id	Integer	主键，自增
customer_name	String(100)	客户名称
product_grade	String(10)	销售等级
quantity	Integer	销售数量
total_price	Float	订单总价
sale_date	DateTime	销售时间

表 4-5 系统设置表 SystemSettings

字段名	类型	说明
id	Integer	主键，自增
key_name	String(50)	设置键名，唯一
value	String(200)	设置值

表 4-6 库存表 Inventory

字段名	类型	说明
grade	String(10)	等级主键

count	Integer	库存数量，非负整数
-------	---------	-----------

5 系统实现

5.1 用户功能模块的实现

5.1.1 用户登录界面

登陆模块用统一的界面风格设计，普通用户通过不同的标签页进入对应账号入口，管理员通过别的途径进行操作。后端收到登录请求之后，会根据账号信息和入口类型来完成角色匹配校验工作，从而保证不同的身份用户可以按照正确的权限去访问。用户退出系统之后，系统会把工作日志结束时间同步到数据库中，给之后的工作时长统计提供基础数据。



图 5-1 登录界面截图

部分关键代码如下：

```
# app.py
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form.get('username')
        password = request.form.get('password')
        login_role = request.form.get('login_role')
        user = User.query.filter_by(username=username).first()

        if user and check_password_hash(user.password, password):
            if user.role != login_role:
                flash('身份不匹配，请切换到对应标签页登录。', 'warning')
                return redirect(url_for('login'))
            login_user(user)
            return redirect(url_for('dashboard'))
    return render_template('login.html')
```

5.1.2 检测历史查询界面

历史记录页面用于展示和追溯已经完成的检测结果。页面集中显示结果图、文件名、板材等级、缺陷类型和检测时间等信息，并提供缺陷关键词筛选功能。由于系统运行后检测记录会不断增加，页面采用分页方式控制单页展示数量，从而提高查询和浏览效率。管理员可以查看全部用户的检测记录，普通用户只能查看本人产生的记录，该设计与系统权限控制逻辑保持一致。

在删除操作中后端加上了权限控制。管理员可以查看、管理所有的检测数据，普通用户只能查看、操作自己产生的记录。这样处理既可以满足多用户系统权限边界的要求，又可以提高历史数据管理的安全性。

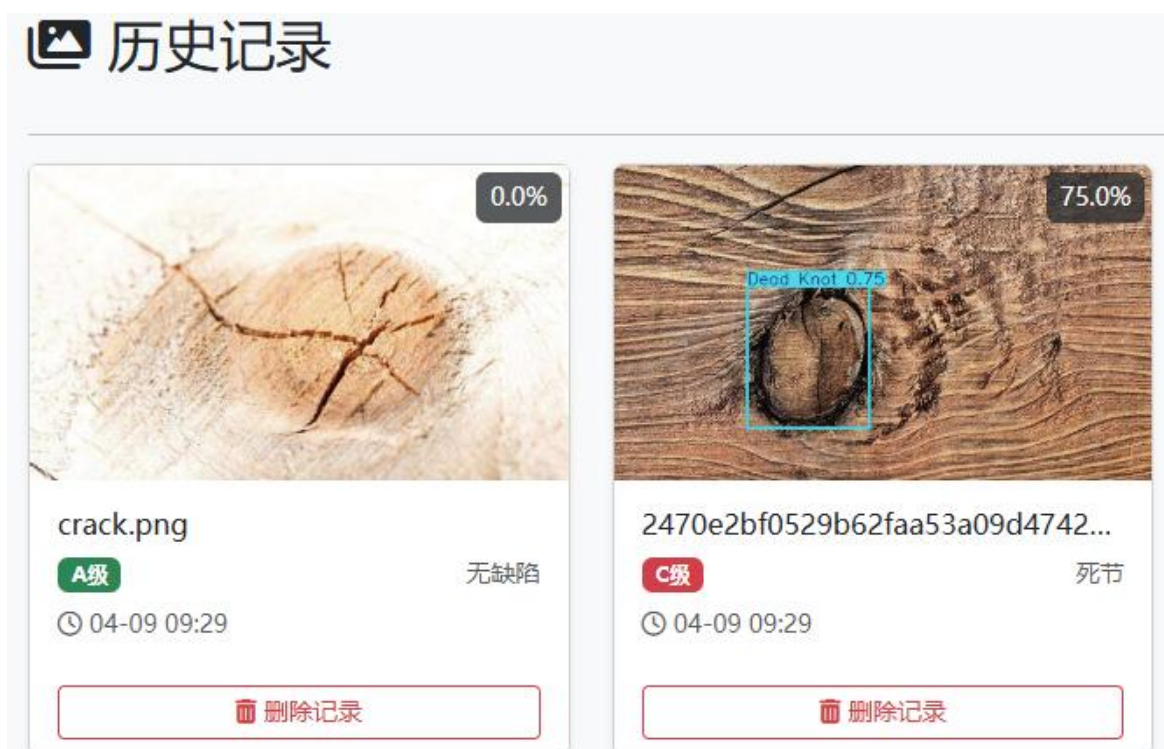


图 5-3 历史记录页面截图

部分关键代码如下：

```
# app.py
@app.route('/history', methods=['GET', 'POST'])
@login_required
def history():
    page = request.args.get('page', 1, type=int)
    search = request.args.get('search', '')
    query = DetectionHistory.query
    if current_user.role != 'admin':
        query = query.filter_by(user_id=current_user.id)
    if search:
        query = query.filter(DetectionHistory.defect_type.contains(search))
    records = query.order_by(
```

```
DetectionHistory.upload_date.desc()
).paginate(page=page, per_page=12)
return render_template('history.html', records=records)
```

5.1.3 日志报表查看界面

日志报表页面主要对系统中已经保存的检测数据和工作日志进行统计、整理与可视化展示。页面展示今日检测量、累计工作时长、缺陷类别占比和近七天检测趋势等内容，使用户能够直观了解系统近期运行情况。该模块既服务于检测结果汇总，也为后续质量分析和管理决策提供数据依据。

数据导出功能使用 `pandas`、`openpyxl` 对检测记录进行整理，生成 Excel 文件供浏览器下载。该功能可以把系统内部的结构化数据转变为可以归档、汇总、二次分析的表格文件，从而提高检测结果的数据利用价值。



图 5-4 日志与报表页面截图

部分关键代码如下：

```
# app.py
@app.route('/logs')
@login_required
def logs():
    today_count = hist_query.filter(
        func.date(DetectionHistory.upload_date) == date.today()
    ).count()

    stats_data = [[k, v] for k, v in counter.items()]
    trend_data = []
    for i in range(6, -1, -1):
        target_date = date.today() - timedelta(days=i)
        count = hist_query.filter(
            func.date(DetectionHistory.upload_date) == target_date
        ).count()
        trend_data.append(count)
```

工作时长统计表由 `WorkLog` 表来提供。用户每次登录时系统就会新增一条日志记录，退出时再补写离开时间；报表页面在统计累计工作时长的时候，会同时计算已结束记录和当前仍然处于登录状态的记录，所以页面上显示的是一个随着使用过程不断变化的统计结果。

该统计逻辑虽然实现方式简单，但是使系统记录的内容不再只是检测次数本身，而是将用户的工作时长也纳入进来，从而使得日志数据更加完整，也为报表分析提供更多的参考信息。

日志与报表

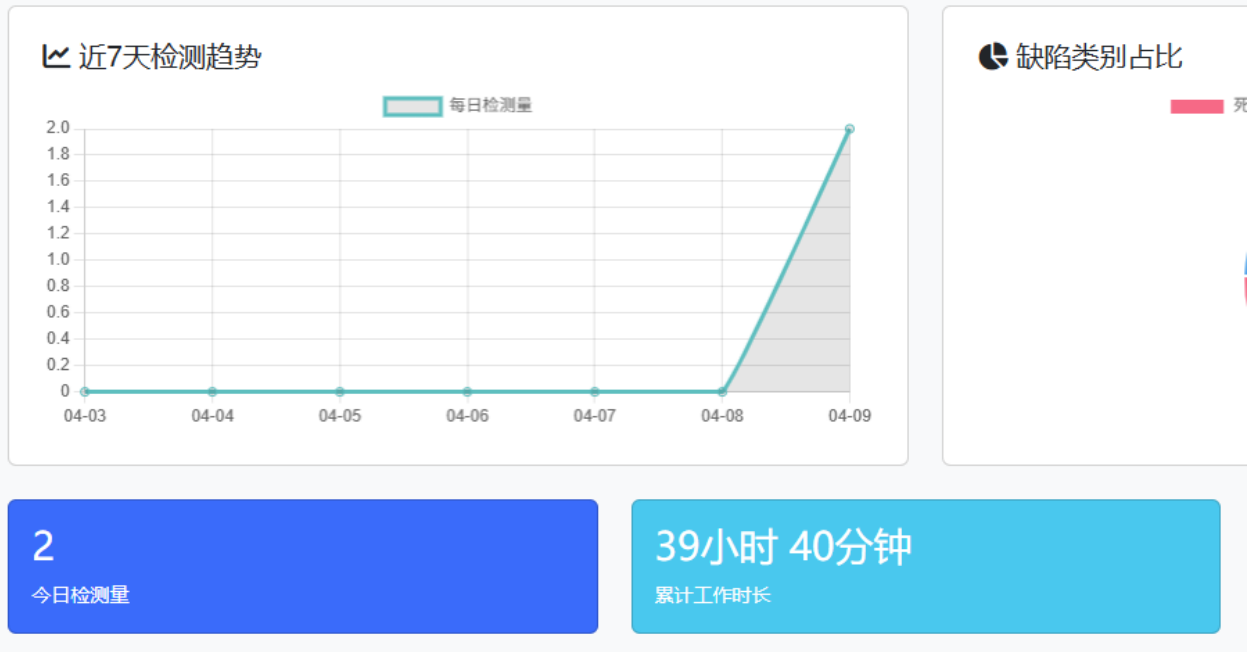


图 5-5 工作时长统计截图

部分关键代码如下：

```
# app.py
total_seconds = 0
for l in log_query.filter(WorkLog.logout_time != None).all():
    total_seconds += (l.logout_time - l.login_time).total_seconds()

if session.get('current_log_id'):
    curr = db.session.get(WorkLog, session['current_log_id'])
    if curr:
        total_seconds += (datetime.now() - curr.login_time).total_seconds()

work_time_str = (
    f"{int(total_seconds // 3600)}小时 "
    f"{int((total_seconds % 3600) // 60)}分钟"
)
```

5.1.4 销售与库存管理界面

销售和库存管理页面在业务上和检测模块是联动的。页面上显示 A、B、C 三种板材库存数量，下方有销售订单录入表单。用户提交订单之后，系统会对对应的等级库存是否足够进行检查，只有库存满足要求的时候才会执行出库操作并记录销售数据，保证销售数据和库存数据的一致性。

销售与库存

21

A级 库存

3

B级 库存

108

C级 库存

新建订单

客户名称

产品等级

A级 (无缺陷)

数量

1

总价

输入金额

出库并销售

销售记录

客户	等级	数量	金额	时间
d	A级	2	100.0	02-15 01:10
ddd	C级	20	600.0	02-15 01:09
d	B级	1	500.0	02-13 13:18
s	C级	5	100.0	02-10 23:51
x	C级	1	20.0	02-10 23:14

图 5-6 销售与库存页面截图

部分关键代码如下:

```
# app.py
@app.route('/sales', methods=['GET', 'POST'])
@login_required
def sales():
    if request.method == 'POST':
        grade = request.form.get('grade')
        qty = int(request.form.get('quantity'))
        inv = db.session.get(Inventory, grade)
        if not inv or inv.count < qty:
            flash(f'{grade} 库存不足!', 'danger')
        else:
            inv.count -= qty
            sale = SalesRecord(
                customer_name=request.form.get('customer'),
                product_grade=grade, quantity=qty
            )
            db.session.add(sale)
```

5.1.5 缺陷图谱与帮助中心界面

帮助中心页面给出活节、死节和裂纹三种典型的缺陷样张，给出相应的定义说明、外观特征和等级判定标准。该页面可以辅助用户理解不同的缺陷之间的差别，使检测结果的说明更加直观，也利于新用户快速掌握系统的业务规则。

缺陷图谱与帮助中心

此处展示了木材表面常见的缺陷类型定义及标准判定样张，供 新员工/新用户 参考学习。

✓ 活节 (Live Knot)



判定等级: **B级 (二等品)**

定义:
由树木的活枝条形成的节子。其组织与周围木材紧密连生，质地坚硬，结构正常。

特征:
颜色通常比周围木材稍深，但没有黑色死皮环绕。它是木材天然特征的一部分，对结构强度影响较小。

✗ 死节 (Dead Knot)



判定等级: **C级 (次品)**

定义:
由树木枯死的枝条形成的节子。其四周不与周围木材连生或部分连生。

特征:
通常被一圈黑色的树皮或腐朽组织包围，质地松软，容易脱落形成空洞（节孔），严重影响板材外观和强度。

⚡ 裂纹 (Crack)



判定等级: **C级 (次品)**

定义:
木材纤维沿纹理方向分离形成的裂隙。

特征:
表现为细长的缝隙。可能由于干燥收缩、外力冲击或生长应力造成。裂纹会破坏木材的完整性，极易导致断裂。

① 说明

- **A级 (优等品):** 表面光洁，无上述任何明显缺陷。
- **B级 (二等品):** 允许存在少量直径较小的活节，但不影响整体强度。
- **C级 (次品):** 存在死节、裂纹或其他严重缺陷，需剔除或降级处理。

图 5-7 帮助中心页面截图

部分关键代码如下：

```
# app.py / templates/help.html
@app.route('/help')
@login_required
def help_center():
    return render_template('help.html')




```

5.2 管理员功能模块的实现

5.2.1 用户管理界面

用户管理模块只对管理员开放，系统可以新增、删除用户账号，在创建用户的时候会对密码进行哈希处理。本模块实现了系统基础的账号管理功能，为后面权限控制、多用户协同使用打下了基础。

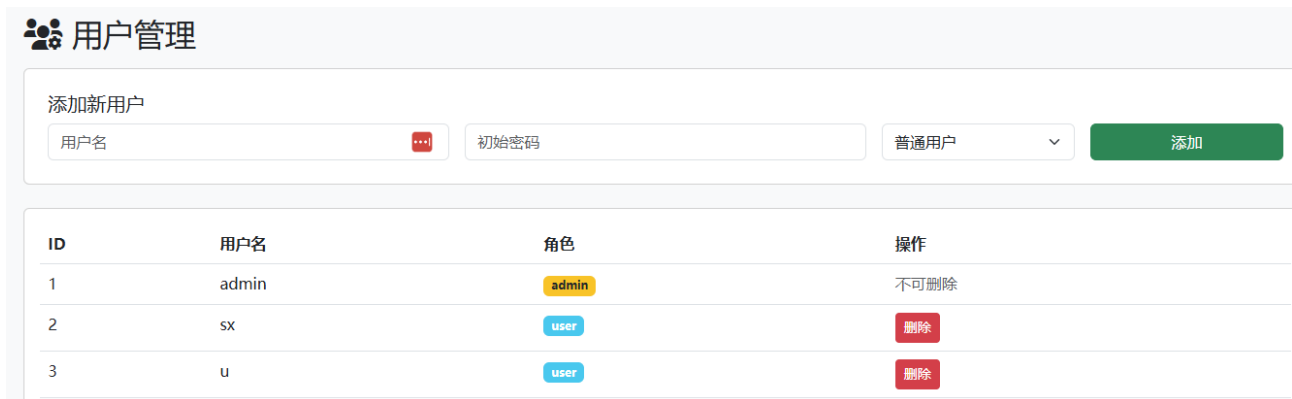


图 5-8 用户管理页面截图

部分关键代码如下：

```
# app.py
@app.route('/admin/users', methods=['GET', 'POST'])
@login_required
def admin_users():
    if current_user.role != 'admin':
        return redirect(url_for('dashboard'))
    if request.method == 'POST':
        action = request.form.get('action')
        if action == 'add':
            new_user = User(
                username=request.form.get('username'),
                password=generate_password_hash(request.form.get('password')),
                role=request.form.get('role')
            )
            db.session.add(new_user)
            db.session.commit()
```

5.2.2 系统设置与模型切换界面

系统设置页面主要是对检测阈值进行调节，对模型文件进行切换。检测阈值会影响模型推理时目标框保留的条件，模型上传功能使管理员可以在不修改程序代码的情况下替换新的模型权重文件，所以该页面可以较好地体现出系统的可维护性以及扩展性。



图 5-9 系统设置页面截图

部分关键代码如下：

```
# app.py
@app.route('/settings', methods=['GET', 'POST'])
@login_required
def settings():
    s = SystemSettings.query.filter_by(key_name='conf_threshold').first()
    s.value = request.form.get('conf_threshold')
    f = request.files.get('model_file')
    if f and f.filename.endswith('.pt'):
        m = SystemSettings.query.filter_by(key_name='current_model').first()
        m.value = secure_filename(f.filename)
```

5.3.1 数据获取与预处理

检测数据获取与预处理是算法检测模块的第一步。系统把本地上传和摄像头采集两种输入方式整合在检测页面中，并通过选项卡进行切换，使用户可以根据实际使用场景选择图片来源。对于本地上传方式，系统允许用户一次选择一张或多张木质板材图片；对于摄像头采集方式，系统通过浏览器调用摄像头画面并截取当前图像用于检测。页面同时增加了本地图片预览功能，便于用户在正式提交检测前确认所选文件是否正确。



在后端处理过程中，系统会先判断上传文件是否存在，再将原始图片保存到服务器指定目录。保存后的图片路径会作为模型推理函数的输入参数，随后由检测模块读取图像文件并完成推理。该处理流程既保证了检测输入来源的灵活性，也为后续结果图生成、检测记录保存和历史追溯提供了基础。

图 5-10 检测页面截图

部分关键代码如下：

```
# templates/detect.html
<ul class="nav nav-tabs mb-3" id="detectTab" role="tablist">
  <li class="nav-item">
    <button class="nav-link active" id="upload-tab"
      data-bs-target="#upload-pane" type="button">
      本地上传
    </button>
  </li>
  <li class="nav-item">
    <button class="nav-link" id="camera-tab"
      data-bs-target="#camera-pane" type="button">
      摄像头采集
    </button>
  </li>
</ul>
```



5.3.2 系统检测与可视化分析

系统检测与可视化分析部分主要说明模型输出结果如何在页面上展示，并如何与系统业务数据形成关联。等级判定不是结果展示阶段附带给出的说明，而是检测流程中的重要业务环节。系统默认无缺陷板材为 A 级，识别到活节为 B 级，检测到裂纹或死节为 C 级。等级结果产生之后，后端会同步更新库存表，使单次检测操作可以和库存管理流程自然衔接。



图 5-12 等级判定与库存联动截图

部分关键代码如下：

```
# app.py
detected_grade = 'A 级'
if res['is_clean']:
    detected_grade = 'A 级'
else:
```

```

for d in res['defects'].split(','):
    zh_name = DEFECT_MAPPING.get(d.strip(), d.strip())
    if zh_name in ['裂纹', '死节']:
        detected_grade = 'C 级'
        break
    elif zh_name == '活节':
        detected_grade = 'B 级'

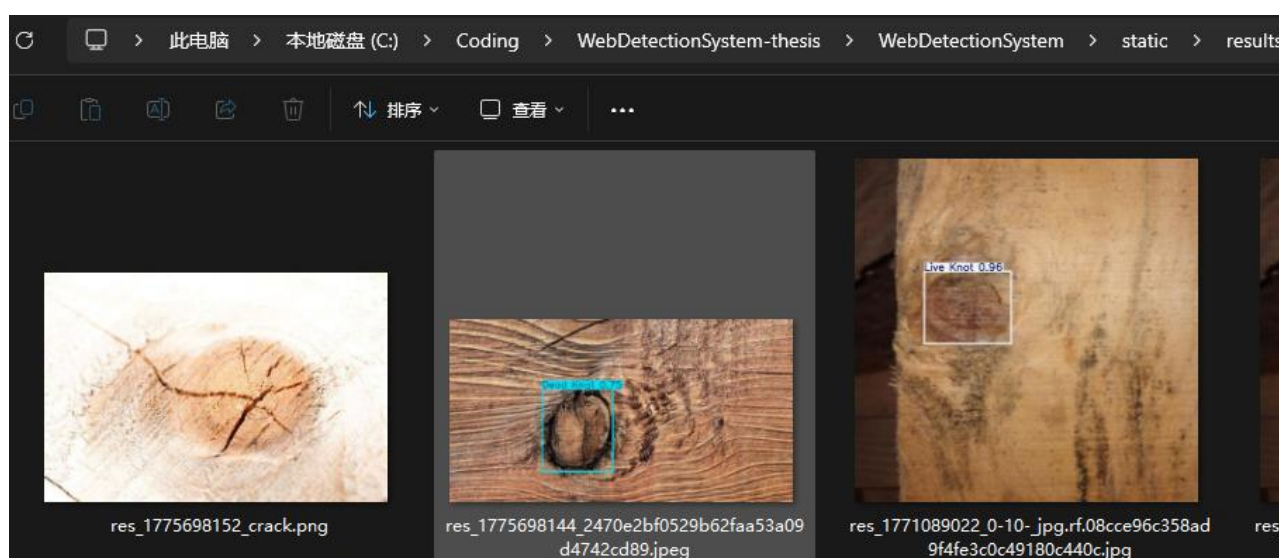
inv = db.session.get(Inventory, detected_grade)
if inv:
    inv.count += 1

```

每完成一次检测，系统都会将结果写入 `DetectionHistory` 表。保存的内容有原始文件名、结果图文件名、缺陷类型、最高置信度、操作用户和等级结果等字段；结果图片单独保存到 `static/results` 目录下，前端页面再用静态资源路径访问。这样就将图片文件和结构化记录分开存贮，方便页面展示以及报表统计之间的联系。

基于此数据保存方式，历史追溯、图片预览、统计报表等都可以建立在同一批检测数据的基础上，减少重复整理数据的工作量，提高系统各个模块之间数据的一致性。

图 5-13 检测结果持久化截图



部分关键代码如下：

```

# app.py
history = DetectionHistory(
    filename=filename,
    result_image=res['filepath'],
    defect_type=final_defect_str,
    confidence=res.get('confidence', 0.0),
    user_id=current_user.id,
    grade=detected_grade
)
db.session.add(history)
results.append(res)
db.session.commit()

```


6 系统测试

6.1 测试目的

系统测试的主要目的就是检验木材表面缺陷智能检测系统在目前的运行环境中，是否能够稳定地完成各项主要业务流程，并且检查系统在正常输入和异常输入情况下所做出的反应。本章测试重点为用户登录、缺陷检测、检测记录保存、日志报表统计、Excel 数据导出、销售出库和后台设置等，用测试进一步证明系统是否达到了毕业设计阶段的基本使用要求。

6.2 测试环境

本次测试环境同系统日常开发环境一致。测试平台使用的是 Windows 操作系统，后端程序运行在本地 Python 虚拟环境里，Web 框架使用的是 Flask，数据库使用的是 SQLite，检测模块调用训练好的模型权重文件进行推理；浏览器端主要是对登录、图片上传、摄像头采集、图表展示、销售管理等页面进行测试。

测试时以普通用户和管理员两种不同的身份进入系统，并按照实际的业务流程检验各个主要的功能模块。除了正常的测试之外，对没有选择图片直接提交检测、角色入口和账号身份不匹配、库存不足时继续提交订单等异常情况进行检查，看系统能否给出明确的提示，并且保持稳定运行。

6.3 测试用例

测试编号	测试内容	预期结果	测试结论
T1	普通用户登录	账号密码正确且角色匹配时成功进入仪表盘	通过
T2	管理员误用普通用户入口登录	系统提示身份不匹配并拒绝登录	通过
T3	上传单张图片检测	返回结果图、缺陷类别、等级和置信度	通过
T4	批量图片检测	多张图片依次返回检测结果并写入历史	通过
T5	摄像头拍照检测	拍照后成功提交并返回检测结果	通过
T6	按缺陷类型查询历史	历史记录支持关键字搜索与分页	通过
T7	导出 Excel 报表	浏览器下载包含检测记录的表格文件	通过
T8	库存不足时提交销售订单	系统阻止出库并提示当前库存	通过
T9	管理员上传新模型	保存成功并在后续检测中优先使用新模型	通过

登录功能测试主要验证系统能否根据用户输入的账号、密码和角色入口完成身份校验。若账号密码正确且角色匹配，系统应进入对应的功能页面；若账号密码错误或角色入口不匹配，系统应给出提示并拒绝登录。具体测试用例如表 6-1 所示。

表 6-1 登录功能测试用例表

6.3.1 正常功能测试截图

正常功能测试围绕系统的主要业务流程展开。测试时先使用正确账号完成登录，再上传木质板材图片进行缺陷检测，随后检查页面是否能够返回结果图、缺陷类别、置信度和板材等级，并确认检测记录是否写入历史记录页面。测试结果表明，系统在正常输入条件下能够完成登录、检测、结果展示和记录保存等核心流程，相关测试结果如图 6-1 所示。

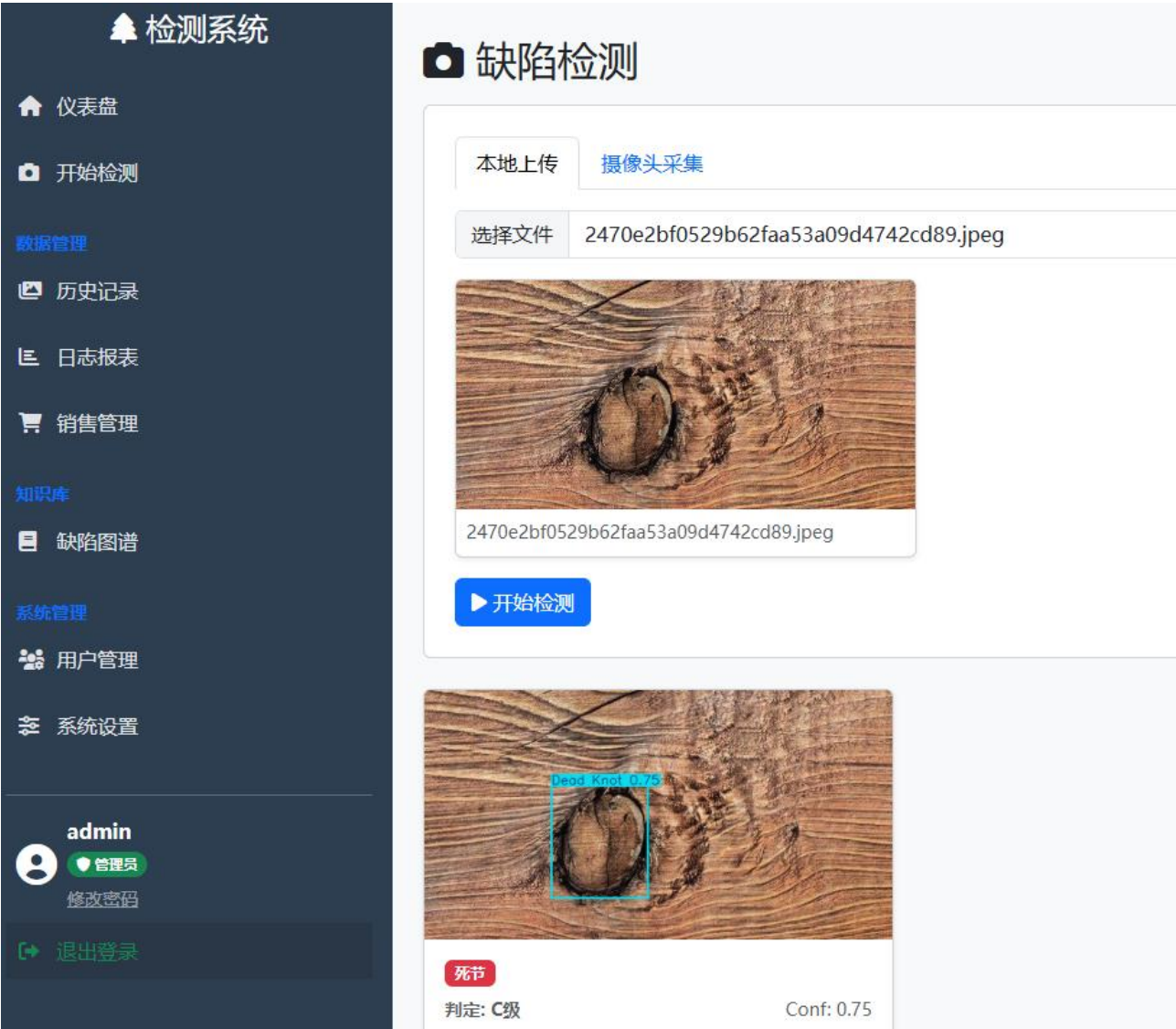


图 6-1 正常功能测试截图

6.3.2 异常功能测试截图

异常功能测试主要用于验证系统在错误输入或业务条件不满足时的处理能力。测试内容包括未选择图片时直接提交检测、普通用户误用管理员入口登录、账号角色不匹配以及库存不足时继续提交销售订单等情况。测试过程中，系统能够返回明确提示并阻止错误操作继续执行，说明系统具备基本的异常处理能力和业务约束能力，相关测试结果如图 6-2 所示。



图 6-2 异常功能测试截图

6.4 测试结论

从综合测试结果可知，系统已经可以比较完整的实现论文前文设计的主要功能，即木材表面缺陷图像检测、等级判定、历史追溯、日志报表统计、库存更新、销售出库和管理员后台管理等业务。整体运行比较稳定，可以满足毕业设计阶段对于系统展示和功能测试的基本需求。

但从测试情况看，系统仍有进一步完善空间。历史记录删除之后没有同步清理对应图片文件，数据库目前还是单机部署的 SQLite，测试工作也只做功能验证，对高并发访问、异常恢复以及长期运行稳定性等方面的覆盖还比较欠缺。这些问题可以作为后面优化、拓展的出发点。

7 总结与展望

本文围绕木质板材表面缺陷检测和业务数据管理需求，设计并实现了一套基于 Python 的 Web 检测系统。系统以 Flask 作为后端框架，使用 SQLite 和 SQLAlchemy 保存用户、检测历史、工作日志、库存、销售记录和系统参数，结合 YOLO 目标检测模型完成裂纹、死节、活节等常见缺陷识别。通过系统分析、结构设计、数据库设计、功能实现和测试验证可

以看出, 论文中的功能划分、业务流程和数据结构均能在实际系统中找到对应实现, 系统设计与开发结果基本一致。

未来可以从三个方面来开展完善工作。第一, 对数据集进行补充, 并对模型进行改进, 从而提高系统对于复杂木纹、光照变化、多缺陷同时出现等条件的识别稳定性; 第二, 将现有的单机运行方式逐步拓展为适合多人使用的部署结构, 扩大系统的使用范围; 第三, 继续完善业务层的功能, 例如图片文件的清理、权限粒度的控制、日志审计、更完整的销售统计。硬件、时间等条件允许的情况下, 系统可以扩大到移动端采集或者现场摄像设备的应用场景。

参考文献

[1]朱咏梅, 李玉玲, 奚峥皓, 等. 注意力可变形卷积网络的木质板材瑕疵识别[J]. 西南大学学报(自然科学版), 2024,46(02):159-169.

[2]万怡, 贺福强, 黄易周, 等. 多功能木质板材新型干燥工艺研究[J]. 林业机械与木工设备, 2023,51(05):34-39.

[3]胡新悦, 杨浩春. 定制家居木质板材表面抗油污性能评价方法探讨[J]. 中国人造板, 2024,31(02):35-38.

[4]薛俊峰. S355J0 板材表面结疤缺陷原因分析[J]. 冶金信息导刊, 2022,59(02):29-32.

[5]王嘉瑜. 常见木质板材燃烧性能研究[J]. 消防界(电子版), 2022,8(15):14-18.

[6]杨艳滨, 任国庆, 汪超. 相控阵超声检测技术在板材中的应用[J]. 无损检测, 2024,46(07):47-53.

[7]杜英国. 氧化铝高压溶出过程管道结疤的特性及清理方法优化[J]. 绿色矿冶, 2023,39(03):45-49+67.

[8]吴志轩, 黄少楠, 黄远民, 等. 一种基于混合纹理特征的板材表面缺陷检测方法[J]. 科学技术创新, 2024,(16):30-33.

[9]Deqiang Z ,Tianqi J ,Xue G , et al.Investigation on magnetic shield thickness of remote field eddy current probes for inspection of ferromagnetic and non-ferromagnetic plates[J].International Journal of Applied Electromagnetics and Mechanics,2023,71(4):325-339.

[10]Mekhtiyev A ,Sarsikeyev Y ,Gerassimenko T , et al.Development of a magnetic activator to protect an electric water heater against scale formation[J].Eastern-European Journal of Enterprise Technologies,2024,6(1):95-102.

[11]陈宇. 基于改进 YOLOX 算法的木质板材表面缺陷在线检测研究[D]. 齐鲁工业大学, 2024.

[12]何涵. 基于改进 YOLOv5 的木质板材封边缺陷检测[D]. 中国科学院大学(中国科学院西安光学精密机械研究所), 2024.

[13]易志浩. 基于机器视觉的木质板材表面缺陷检测方法研究[D]. 佛山科学技术学院, 2024.

[14]徐浩, 夏振平, 林李兴, 等. 多特征融合的在线板材表面缺陷检测方法研究[J]. 苏州科技大学学报(自然科学版), 2024,41(01):76-84.

[15]凌嘉欣, 谢永华. 残差神经网络模型在木质板材缺陷分类中的应用[J]. 东北林业大学学报, 2021,49(08):111-116.

致谢

本论文的完成，离不开老师的悉心指导、同学的热心帮助以及家人的默默支持。在此，我向所有关心和帮助过我的人表示最衷心的感谢。首先，我诚挚地感谢我的指导教师。在整个毕业设计过程中，老师从课题方向、系统架构、算法设计到论文结构都给予了我细致的指导。在基于 Python 开发木质板材表面缺陷检测系统时，面对图像预处理、缺陷特征提取、深度学习检测等技术难点，老师总能及时为我指点迷津，使我能够顺利完成系统开发与论文撰写。老师严谨的治学精神与认真负责的态度，让我深受启发，也将成为我未来学习和工作中的宝贵财富。

感谢同窗好友在学习与项目开发中给予我的支持与陪伴，我们相互探讨、共同进步，让毕业设计的过程充实而温暖。感谢家人一直以来的理解、鼓励与支持，是他们的关爱让我能够安心完成学业。感谢所有参与论文评审的专家教授，感谢你们提出的宝贵修改意见，使本论文更加完善。由于本人水平有限，系统与论文中仍存在不足之处，恳请各位老师批评指正。在今后的道路上，我将继续努力学习，不断提升自我，不负师长与家人的期望。